

stackproofs: Private proofs of stack and contract execution using PROTOGALAXY

Liam Eagen¹, Ariel Gabizon², Marek Sefranek³, Patrick Towa², and Zachary J. Williamson²

¹Alpen Labs

²Aztec Labs

³TU Wien

December 6, 2024

Abstract

The goal of this note is to describe and analyze a simplified variant of the zk-SNARK construction used in the Aztec protocol. Taking inspiration from the popular notion of Incrementally Verifiable Computation [Val08] (IVC) we define a related notion of *Repeated Computation with Global state* (RCG). As opposed to IVC, in RCG we assume the computation terminates before proving starts, and in addition to the local transitions some global consistency checks of the whole computation are allowed. However, we require the space efficiency of the prover to be close to that of an IVC prover not required to prove this global consistency. We show how RCG is useful for designing a proof system for a private smart contract system like Aztec.

1 Introduction

Incrementally Verifiable Computation (IVC) [Val08] and its generalization to Proof Carrying Data (PCD) [CT10] are useful tools for constructing space-efficient SNARK provers [BCCT12]. In IVC and PCD we always have an acyclic computation. However code written in almost any programming language *is* cyclic in the sense of often relying on internal calls – we start from a function A , execute some commands, go into a function B , execute its commands, and go back to A . When making a SNARK proof of such an execution, we typically linearize or “flatten” the cycle stemming from the internal call, in one of the following two ways.

1. The monolithic circuit approach: We “inline” all internal calls (as well as loops) into one long program without jumps.

2. The VM approach: Assume the code of A, B is written in some prespecified instruction set. The program is executed by initially writing the code of A, B into memory, and loading from memory and executing at each step the appropriate instruction according to a program counter. For example, the call to B is made by changing the counter to that of the first instruction of B . To prove correctness of the execution, all we need is a SNARK for proving correctness of a certain number of steps of a machine with this instruction set, and some initial memory state.

The second approach is more generic, while the first offers more room for optimization, so we'd want to use it in resource-constrained settings, e.g. client-side proving.

However, what if we're in a situation where A and B have already been "SNARKified" separately? Namely, there is a verification key attached to each one, and we are expected to use these keys specifically. This is what happens in the Aztec system.

The Aztec private contract system Similar to Ethereum we have contracts, and the contracts have functions.¹ A function in a contract can internally call a different function in the same or a different contract. Moreover, while writing the code for the different functions and compiling them to circuits, we can't predict what function will be internally called by a given contract function. For example, a "send token" function could have an internal call to an "authorize" function. But the call to "authorize" need not be tied to a specific implementation and consequently, to a specific verification key – as different token holders are allowed to set their own "authorize" function.

The goal of the Aztec system is to enable constructing zero-knowledge proofs of such contract function executions. For this purpose, a contract is deployed by

1. Computing a verification key for each function of the contract.
2. Adding a commitment to the verification keys of the contract in a global "function tree". More accurately, a leaf of this tree is a hash of the contract address with a Merkle root of a tree whose leaves are the verification keys of that contract's functions.

Dealing with global state The global state of the Aztec system is described by a set of notes, which are simply values in a field $\mathbb{F}$. Each note belongs to a certain contract. While running, a function can read, add or delete notes belonging to its contract. We can thus think of the notes as global variables shared between the different functions.

Assume all functions in this system return `accept` or `reject`. (We can always move the output into the arguments if a function is not of this form.) Here's a natural way to prove the mentioned execution: Put the arguments to B in the public inputs of both the circuits of A and B . Verify the proofs π_A, π_B for A, B ; and check via the public inputs the same value was used in both proofs for the arguments of B .

This however, doesn't yet deal with the notes. During execution, note operations happened in a certain order. We can thus assign a *counter* equal to one for the first

¹Detailed documentation of the Aztec protocol can be found [here](#).

operation and increment the counter with each operation. We then need to check, for example, that if a note was read with a certain counter, it was indeed added with a smaller counter. We can include a description of the note operations performed by a function in the inputs of its circuit. This description will contain the operation type (`add`, `del`, `read`), the note value, and the counter. The issue is, what if A is reading a note that was added in the internal call to B ? Checking the existence of an `add` operation with smaller counter requires a constraint *between* the inputs of both circuits. And for an execution consisting of more calls, this constraint can involve any two circuits in the call tree.

This brings us to the notion of *Repeated Computation with Global state* (RCG). In RCG we have a transition predicate taking us from one state to the next. We wish to prove we know a sequence of witnesses taking us from a legal initial state to a certain publicly known final state. This might remind the reader of the popular notion of *incrementally verifiable computation* (IVC). There are two differences.

- In RCG we are not interested in “incremental” proofs of one step, only in proofs for a whole sequence of transitions ending in a desired final state.
- In RCG we also have a *final predicate* checking a joint consistency condition between witnesses from all iterations.

One could ask, why not *only* have a final predicate that includes the transition checks? In other words, a monolithic circuit for the whole computation. The point is that in our use case the final predicate is applied to small parts of each iteration’s witness – namely the note operations. As a result, the decomposition into a transition and final predicate can facilitate obtaining better prover efficiency, especially in terms of prover space. Roughly, we’ll require space sufficient for storing the inputs to the final predicate, in addition to the space required to prove a single transition.

1.1 Related work

Recent work [Sou23, NDC⁺24] as well as ongoing work [Gro24] uses folding schemes [BCL⁺21, KST21] to break up proving statements about large computation into smaller statements. The objective being reducing prover memory and/or improving prover parallelism. These works have not formally defined a notion like RCG, and rather use the IVC terminology. We believe the RCG framework may be better suited for capturing the properties of these constructions. In terms of the concrete constructions, there are overlaps with this work, notably a two-pass over part of the witness to generate a random challenge. As alluded to earlier, one distinction is that these works start with a computation that is already linear, while here we start with cyclic computation and must “compile” it into a “linear” statement.

1.2 Overview of the paper

In Section 2, we go over terminology and preliminaries. Notably, in Section 2.2 we introduce the *Zero-Testing Assumption* from [LS24] that enables us to avoid heuris-

tic assumptions as commonly happens in IVC papers due to the need of instantiating the random oracle. In Section 3, we introduce our execution model involving record operations and execution trees. In Section 4, we introduce the notion of Repeated Computation with Global state, and show how to reduce computations in the model of Section 3 to RCG. In Section 5, we show how “log-derivative methods” [Eag22, Hab22] can capture global consistency of operations as defined in previous sections. In Section 6, we show how the **PROTOGALAXY** folding scheme [EG23] can be proven secure under the ZTA (without the need of a random oracle). In Section 8, we present our main zk-SNARK construction. The main ingredient is a “folding to IVC” reduction similar to the ones in [BCL⁺21, KST21]. In Section 9, we outline how RCG can be implemented for a more general set of functions.

A reader wanting to get a feeling of the main construction ideas might want to focus first on Sections 3–5.

2 Preliminaries

2.1 Terminology and Conventions

We assume all algorithms described receive as an implicit parameter the security parameter λ . Similarly, all integer parameters in the paper are implicitly functions of λ , and of size at most $\text{poly}(\lambda)$ unless explicitly stated otherwise.

Whenever we use the term *efficient*, we mean an algorithm running in time $\text{poly}(\lambda)$. Furthermore, we assume an *object generator* $\mathcal{O}$ that is run with input λ before all protocols, and returns all fields and groups used. Specifically, in our protocol $\mathcal{O}(\lambda) = (\mathbb{F}, \mathbb{G}, g, r_{\text{cm}})$ where

- $\mathbb{F}$ is a field of **prime** size $r = \lambda^{\omega(1)}$.
- $\mathbb{G}$ is a group of size r .
- g is a uniformly chosen generator of $\mathbb{G}$.
- r_{cm} is a uniformly chosen key for a collision-resistant homomorphic commitment scheme $\text{cm} : \mathbb{F}^M \rightarrow \mathbb{G}$.

We usually let the λ parameter be implicit, e.g. write $\mathbb{F}$ instead of $\mathbb{F}(\lambda)$. We denote by $\mathbb{F}_{<d}[X]$ the set of univariate polynomials over $\mathbb{F}$ of degree smaller than d . We write $\mathbb{G}$ additively. *All probabilistic statements in the paper are additionally over the randomness of $\mathcal{O}$.*

We often denote by $[n]$ the integers $\{1, \dots, n\}$. We use the acronym e.w.p. for “except with probability”; i.e. e.w.p. γ means with probability *at least* $1 - \gamma$.

2.2 Zero-Testing Assumption

Throughout this paper, we’ll make use of a variant of the *Zero-Testing Assumption* (ZTA) from [LS24] given in Definition 2.1 as well as its generalization, the *t-variate Zero-Testing Assumption*, we introduce in Definition 2.2.

Definition 2.1. Fix $\text{cm} : \mathbb{F}^M \rightarrow K$, hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{F}$, and integer d . Fix the family of functions $\mathcal{D}$. We say the tuple (D, x, τ) is a degree d -relation for $(\mathcal{D}, \mathcal{H}, \text{cm})$ if

1. $D \in \mathcal{D}$.
2. $f(X) := D(x, \tau)$ is a non-zero element of $\mathbb{F}_{\leq d}[X]$.
3. Setting $z := \mathcal{H}(\text{cm}(x), \tau)$, we have $f(z) = 0$.

The Zero-Testing Assumption (ZTA) for $(\mathcal{D}, \mathcal{H}, \text{cm}, d)$ states that for any efficient $\mathcal{A}$, the probability that $\mathcal{A}$ outputs a degree d -relation for $(\mathcal{D}, \mathcal{H}, \text{cm})$ is $\text{negl}(\lambda)$.

Definition 2.2 (t -variate ZTA). Fix $\text{cm} : \mathbb{F}^M \rightarrow K$, hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{F}$, and integer d . Fix the family of functions $\mathcal{D}$. We say the tuple (D, x, τ) is a (t, d) -relation for $(\mathcal{D}, \mathcal{H}, \text{cm})$ if

1. $D \in \mathcal{D}$.
2. $f(X_1, \dots, X_t) := D(x, \tau)$ is a non-zero element of $\mathbb{F}_{\leq d}[X_1, \dots, X_t]$.
3. Setting $z_i := \mathcal{H}(\text{cm}(x), \tau, i)$ for $i \in [t]$, we have $f(z_1, \dots, z_t) = 0$.

The t -variate Zero-Testing Assumption (ZTA) for $(\mathcal{D}, \mathcal{H}, \text{cm}, d)$ states that for any efficient $\mathcal{A}$, the probability that $\mathcal{A}$ outputs a (t, d) -relation for $(\mathcal{D}, \mathcal{H}, \text{cm})$ is $\text{negl}(\lambda)$.

We prove that the univariate ZTA implies the t -variate ZTA.

Lemma 2.3. Fix a family of functions $\mathcal{D}$ whose outputs are polynomials in $\mathbb{F}_{\leq d}[X_1, \dots, X_t]$. Let $\mathcal{D}_t$ be a family of functions to be defined in the proof. Then the univariate ZTA for $(\mathcal{D}_t, \mathcal{H}, \text{cm}, d)$ implies the t -variate ZTA for $(\mathcal{D}, \mathcal{H}, \text{cm}, d)$ and $t = \text{poly}(\lambda)$.

Proof. Let $\mathcal{A}$ be an adversary against the t -variate ZTA that outputs the (t, d) -relation (D, x, τ) for $(\mathcal{D}, \mathcal{H}, \text{cm})$. We construct the adversary $\mathcal{A}'$ against the univariate ZTA that outputs a degree- d relation for $(\mathcal{D}_t, \mathcal{H}, \text{cm})$, where $\mathcal{D}_t$ will be the union of all the functions D_i defined in the following.

Write $f(X_1, \dots, X_t) := D(x, \tau)$ as a polynomial in X_t over $\mathbb{F}[X_1, \dots, X_{t-1}]$:

$$f(X_t) = \sum_{i=0}^d C_i(X_1, \dots, X_{t-1}) X_t^i.$$

For $i \in [t]$, denote $z_i := \mathcal{H}(\text{cm}(x), \tau, i)$. Suppose first that $f(z_1, \dots, z_{t-1}, X_t) \not\equiv 0$. Then $\mathcal{A}'$ can output the degree- d relation (D_t, x, τ_t) , where D_t is the function that computes $f_t(X) := f(z_1, \dots, z_{t-1}, X)$ given x and $\tau_t := (\tau, t)$, deriving $z_1, \dots, z_{t-1}$ via $\mathcal{H}$ as part of its operation. Note that $f_t(z_t) = 0$ with $z_t = \mathcal{H}(\text{cm}(x), \tau_t)$.

Otherwise, there is a non-zero polynomial $C_i \in \mathbb{F}_{\leq d}[X_1, \dots, X_{t-1}]$ which satisfies $C_i(z_1, \dots, z_{t-1}) = 0$. If $C_i(z_1, \dots, z_{t-2}, X_{t-1}) \not\equiv 0$, $\mathcal{A}'$ can output the degree- d relation

(D_{t-1}, x, τ_{t-1}) , where D_{t-1} is the function that computes $f_{t-1}(X) := C_i(z_1, \dots, z_{t-2}, X)$ given x and $\tau_{t-1} := (\tau, t-1)$.

Recursively define degree- d relations (D_i, x, τ_i) until $i = 1$ and we have a univariate non-zero polynomial $C'_j \in \mathbb{F}_{\leq d}[X_1]$ with $C'_j(z_1) = 0$. In this base case, $\mathcal{A}'$ can output the degree- d relation (D_1, x, τ_1) , where D_1 is the function that computes $f_1(X) := C'_j(X)$ given x and $\tau_1 := (\tau, 1)$, finishing the proof. $\square$

3 The execution model

We present a formal framework for describing function executions enabling both internal function calls and global state.

We begin in Section 3.1 by introducing *record operations* which is our specific notion of operating on a global state. Record operations keep track of the computation steps at which records (roughly corresponding to notes in the introduction) were added, read and deleted. We then define what it means for a set of such operations to be *consistent*. For example, we want to enforce that a record can be deleted only if it was first added.

The eventual goal is to prove that a certain set of records is the output of a function execution (where that execution includes the function's internal calls). The proof should not reveal any information about which function was executed, as long as it belongs to a pre-defined set of legal functions, or about its arguments. To do so, it is sufficient to prove knowledge of an execution tree with the initial function at the root, the functions it calls at its children nodes and so forth. Section 3.2 defines a relation which formally captures what it means for an individual function to accept on its arguments, and Section 3.3 formally defines such execution trees.

Remark 3.1. *For simplicity, and in contrast to the introduction, we don't explicitly discuss contracts. We only model functions operating on a shared global state. As in Aztec each function can only operate on notes of its contract, this corresponds to a system with one contract. Capturing the general system mainly requires modelling the restriction that a function is only operating on the subset of notes belonging to its contract.*

3.1 Record operations

Records are pairs (v, c) , where $v \in \mathbb{F}$ is the *value*, and $c \in [M]$ is a *counter*. A *record operation* has one of the following forms:

- (add, v, c) ,
- (del, v, c_v, c) ,
- (read, v, c_v, c) .

Here $v \in \mathbb{F}$ is a value and $c, c_v \in [M]$ are counters. c is interpreted as the counter of the current operation. c_v is interpreted as the counter of the operation where the value was added in the case of a *read* or *del* operation.

We say a sequence $\mathcal{O}$ of record operations of size M is *consistent* if

1. The counter values c are distinct in all elements of $\mathcal{O}$, and as a set equal to $[M]$.
2. The c_v fields in all del operations $(\text{del}, v, c_v, c) \in \mathcal{O}$ are distinct.
3. If $(\text{del}, v, c_v, c) \in \mathcal{O}$, then $c_v < c$ and $(\text{add}, v, c_v) \in \mathcal{O}$.
4. If $(\text{read}, v, c_v, c) \in \mathcal{O}$, then $c_v < c$ and $(\text{add}, v, c_v) \in \mathcal{O}$.

Let V be a set of records. We say $\mathcal{O}$ *has output* V if:

- $\mathcal{O}$ is consistent.
- $V = \{(v, c) \mid (\text{add}, v, c) \in \mathcal{O} \wedge \forall c' \in [M], (\text{del}, v, c, c') \notin \mathcal{O}\}$. In words, V is the set of values that were added and not deleted.

3.2 The Plonkish relation

Now we introduce a relation $\mathcal{R}_{app}$ describing the individual function executions tailored to make it convenient to later discuss an execution of a *sequence* of functions calling each other. The executed function is represented in the instance by a single group element f . In the terminology of [GWC19], f is a commitment to the permutation and selector values of a specific $\mathcal{PlonK}$ circuit. In particular, $\mathcal{R}_{app}$ is a “universal” Plonkish relation where the circuit is not fixed but chosen in the instance.

Additionally, the instance adheres to a form containing both the record operations and the details of the inner calls of the individual function execution. We stress however, that the *interpretation* of these values as record operations and inner calls, only happens in the next section when we discuss valid executions trees; and doesn’t manifest in the definition of $\mathcal{R}_{app}$. Some choices of constants – like args being of size four – are arbitrary.

We fix a polynomial $G : \mathbb{F}^8 \rightarrow \mathbb{F}$, and integers N, n that are implicit parameters in the following definition of relation $\mathcal{R}_{app}$.

$\mathcal{R}_{app}$ consists of all pairs $(\mathbf{r}, \mathbf{w})$ having the form

- $\mathbf{r} = (f, \text{args}, c, f_1, \text{args}_1, f_2, \text{args}_2, \mathcal{O})$ where $f, f_1, f_2 \in \mathbb{G}, \text{args} \in \mathbb{F}^4, c \in \{0, 1, 2\}$;
- $\mathbf{w} = (w_f, \omega)$ where
 - $w_f = (S_1, \dots, S_4, q_1, \dots, q_4)$, where $S_j \in [|\mathbf{r}| + N]^n, q_j \in \mathbb{F}^n$ for each $j \in [4]$
 - $\omega \in \mathbb{F}^N$

such that

1. Setting $x = (\mathbf{r}, \omega)$, for all $i \in [n]$

$$G(q_{1,i}, \dots, q_{4,i}, x_{S_{1,i}}, \dots, x_{S_{4,i}}) = 0.$$

2. $f = \text{cm}(w_f)$.

3.3 Valid execution trees

By an *execution tree of length n* we mean a binary tree T with n vertices, whose nodes are labeled by pairs $(\mathbf{r}, \mathbf{w})$. Let F be a set of elements of $\mathbb{G}$. Given such T we say it is a *valid execution of length n with function set F and output V* if

1. For each $n \in T$, its label $(\mathbf{r}, \mathbf{w})$ is in $\mathcal{R}_{app}$.
2. For each $n \in T$, let $(\mathbf{r}, \mathbf{w})$ be its label. Let $\mathbf{r} = (f, \text{args}, c, f_1, \text{args}_1, f_2, \text{args}_2, \mathcal{O})$. Then
 - $f \in F$.
 - The number of its children is c .
 - For $i \in [c]$, let $(f^i, \text{args}^i, c^i, f_1^i, \text{args}_1^i, f_2^i, \text{args}_2^i, \mathcal{O}^i)$ denote the first component of the label of n 's i -th child. Then $f_i = f^i$ and $\text{args}_i = \text{args}^i$.
 - Let $\mathcal{O}$ be the multi-set union of $\mathbf{r}.\mathcal{O}$ over all nodes' labels $(\mathbf{r}, \mathbf{w})$. Then $\mathcal{O}$ has output V .

Given a set of group elements F , say it has *Merkle root $\mathbf{r}$* if $\mathbf{r}$ is the root of a Merkle tree with the elements of F at the leaves using some pre-determined encoding.

We define a relation $\mathcal{R}_{exec}$ capturing knowledge of an execution of bounded length with a certain output set of records. $\mathcal{R}_{exec}$ consists of the pairs (x_{exec}, w_{exec}) of the form

- $x_{exec} = (\mathbf{r}, C, V)$,
- $w_{exec} = (n, T)$,

such that $n \leq C$, and T is a valid execution tree of length n with function set F having Merkle root $\mathbf{r}$, and output set V .

4 Repeated Computation with Global state

Motivated by space-efficient proofs for $\mathcal{R}_{exec}$, we introduce the notion of *Repeated Computation with Global state* (RCG). RCGs enable us to deal separately with the local consistency of iterative steps of a transition function, and over-all consistency of a global state consisting of a part of each iteration's witness. We first define the general notion, and then in Section 4.1 show how to capture valid execution trees with it.

Defining RCG relations An RCG relation is defined by a pair of functions (F, ℓ) . We call $F(Z, W, Z^*, S) \rightarrow \{\text{accept}, \text{reject}\}$ the *transition predicate*, and $\ell(Z, S_1, \dots, S_n, V) \rightarrow \{\text{accept}, \text{reject}\}$ the *final predicate*. We informally think of

- Z as the public input and W as the private input of F .
- Z^* as the output of F (although the actual output is $\{\text{accept}, \text{reject}\}$).
- S as the part of the private input that will be used in the final predicate.

The relation $\mathcal{R}_{F,\ell}$ is the set of pairs (x, w) with

- $x = (z_{final}, C, V)$,
- $w = (n, z = (z_0, \dots, z_n), w = (w_1, \dots, w_n), s = (s_1, \dots, s_n))$

such that

1. $z_0.\text{init} = \text{true}$.
2. $z_n = z_{final}$.
3. $n \leq C$.
4. For each $i \in [n]$, $F(z_{i-1}, w_i, z_i, s_i) = \text{accept}$.
5. $\ell(z_n, s_1, \dots, s_n, V) = \text{accept}$.

We say a zk-SNARK for $\mathcal{R}_{F,\ell}$ is *space-efficient* if, given s and streaming access to z and w , $\mathbf{P}$ requires space $O(|F| + |s| + \lambda \log n)$. Here $|F|$ is defined as $\mathcal{M} + n'$ where $f : \mathbb{F}^{\mathcal{M}} \rightarrow \mathbb{F}^{n'}$ is the **PROTOGALAXY** constraint function representing F (see Section 6).

4.1 Valid executions as RCGs

We define a specific RCG relation $\mathcal{R}_{F,\ell}$ capturing valid execution trees as defined in Section 3.3. Loosely speaking, the transition function F will update a call stack of functions yet to be executed, and execute the function that is at the top of the stack. The final predicate ℓ will check the union of record operations from all iterations is consistent.

More precisely, define the function $F(Z, W, Z^*, S) \rightarrow \{\text{accept}, \text{reject}\}$ as follows.

- $Z = (g, \mathbf{r}, \text{init})$ where g is a stack of elements of the form (f, args) , $\mathbf{r}$ is a root of a Merkle tree, and init is a boolean.
- $Z^* = (g^*, \mathbf{r}^*, \text{init}^*)$ has the same form.
- $W = (\mathbf{p}, \mathbf{x}, \mathbf{w})$.
- S is a set of record operations.

Under this notation, $F(Z, W, Z^*, S) = \text{accept}$ if and only if

1. If $\text{init} = \text{true}$, g contains exactly one element.
2. Setting $\mathbf{r} = (\mathbf{x}, S)$, we have $(\mathbf{r}, \mathbf{w}) \in \mathcal{R}_{app}$.
3. Denoting $g[0] = (f, \text{args})$, we have $f = \mathbf{x}.f$ and $\text{args} = \mathbf{x}.\text{args}$.
4. $\mathbf{p}$ is a Merkle path from f to $\mathbf{r}$.

5. $\mathbf{r} = \mathbf{r}^*$.

6. g^* is the result of popping (f, args) from g and then pushing the $\mathbf{r}.c$ elements $(\mathbf{r}.f_i, \mathbf{r}.args_i)$ for $i \in [\mathbf{r}.c]$.

Denote by g_{empty} the empty stack. We define $\ell(z_n, s_1, \dots, s_n, \mathbf{V})$ to output **accept** if and only if

1. $z_n.g = g_{\text{empty}}$,
2. Defining $\mathcal{O}$ as the multi-set union of $s_1, \dots, s_n$, it is a well-formed set of record operations with output $\mathbf{V}$.

We show that proving knowledge of a witness for an instance of $\mathcal{R}_{\text{exec}}$ can be reduced to proving knowledge of a witness for an instance of $\mathcal{R}_{F, \ell}$.

Lemma 4.1. *There is an efficiently computable and efficiently invertible map φ such that the following holds. Let $\mathbf{F}$ be a set of function commitments with Merkle root $\mathbf{r}$. Fix positive integers n, C with $n \leq C$. Define $z_{\text{final}} = (g_{\text{empty}}, \mathbf{r}, \text{false})$. Let $\mathbf{T}$ be an execution tree of length n .*

Then $((\mathbf{r}, C, \mathbf{V}), (n, \mathbf{T})) \in \mathcal{R}_{\text{exec}}$ if and only if $((z_{\text{final}}, C, \mathbf{V}), \varphi(n, \mathbf{T})) \in \mathcal{R}_{F, \ell}$.

Proof. We describe the operation of φ . Given $\mathbf{T}$ of length n , let $(\mathbf{r}_1, \mathbf{w}_1), \dots, (\mathbf{r}_n, \mathbf{w}_n)$ be the labels of its nodes according to DFS order. Define a sequence of stacks $g_0, \dots, g_n$ according to the sequence of labels.

Namely, g_0 is the stack containing only $(\mathbf{r}_1.f, \mathbf{r}_1.args)$. And for each $i \in [n]$, g_i is the stack obtained by popping $g_{i-1}[0]$ and pushing $(\mathbf{r}_i.f_j, \mathbf{r}_i.args_j)$ for $j \in [\mathbf{r}_i.c]$. Now, define $z_0 = (g_0, \mathbf{r}, \text{true})$ and for each $i \in [n]$, $z_i = (g_i, \mathbf{r}, \text{false})$.

To proceed we need to refer to the record operations in each instance separately. For this purpose, for each $i \in [n]$ denote $\mathbf{r}_i = (\mathbf{x}_i, \mathcal{O}_i)$. For each $i \in [n]$, let $\mathbf{p}_i$ be the path from $\mathbf{r}_i.f$ to $\mathbf{r}$. Define for each $i \in [n]$, $w_i = (\mathbf{p}_i, \mathbf{x}_i, \mathbf{w}_i)$, $s_i = \mathcal{O}_i$. Finally set $z = (z_0, \dots, z_n)$, $w = (w_1, \dots, w_n)$, $s = (s_1, \dots, s_n)$ and $\varphi(n, \mathbf{T}) = (n, z, w, s)$. Given this definition of φ , the statement of the lemma is straightforward to check. $\square$

5 Removing the global state via rational identities

We give rational identities which are equivalent to the consistency of record operations as formally defined in Section 3.1. They arise from ideas similar to those used in “log-derivative lookups” [Eag22, Hab22]. Using these identities, we then define a new RCG relation $\mathcal{R}_{F^*, \ell^*}$ capturing valid execution trees, i.e., $\mathcal{R}_{\text{exec}}$. The advantage of $\mathcal{R}_{F^*, \ell^*}$ over $\mathcal{R}_{F, \ell}$ from Section 4.1 is that the final predicate is “trivial” in the sense of only depending on the output of the final iteration. As we’ll see in Section 8, this makes constructing a zk-SNARK for it more convenient.

Claim 5.1. *Assume $\mathbb{F}$ has characteristic larger than $M+1$. Let $\mathbf{V}$ be a set of records and $\mathcal{O} = \{(op_i, \mathbf{v}_i, \mathbf{c}_{\mathbf{v}_i}, \mathbf{c}_i)\}_{i \in [M]}$ be a set of record operations (defining $\mathbf{c}_{\mathbf{v}_i} = 0$ when $op_i = \text{add}$) with $\mathbf{c}_{\mathbf{v}_i} < \mathbf{c}_i$ for all $i \in [M]$. Then $\mathcal{O}$ has output $\mathbf{V}$ if and only if the following rational function identities hold:*

1.

$$\sum_{(\mathbf{v}, \mathbf{c}) \in \mathbf{V}} \frac{1}{X + \mathbf{v}Y + \mathbf{c}} = \sum_{i \in [M], op_i = \text{add}} \frac{1}{X + \mathbf{v}_i Y + \mathbf{c}_i} - \sum_{i \in [M], op_i = \text{del}} \frac{1}{X + \mathbf{v}_i Y + \mathbf{c}_{\mathbf{v}_i}}.$$

2. For some $m \in \mathbb{F}^M$, we have

$$\sum_{i \in [M], op_i = \text{add}} \frac{m_i}{X + \mathbf{v}_i Y + \mathbf{c}_i} = \sum_{i \in [M], op_i = \text{read}} \frac{1}{X + \mathbf{v}_i Y + \mathbf{c}_{\mathbf{v}_i}}.$$

3.

$$\sum_{i \in [M]} \frac{1}{X + \mathbf{c}_i} = \sum_{i \in [M]} \frac{1}{X + i}.$$

Proof. We focus on the only if direction. That is, if $\mathcal{O}$ doesn't have output $\mathbf{V}$ one of the three identities should not hold. Let $R_{v,c}(X, Y) := \frac{1}{X + vY + c}$. The main fact we use is that the rational functions $\{R_{v,c}\}_{(v,c) \in \mathbb{F}^2}$ are linearly independent. Thus, $\sum_{v,c} a_{v,c} R_{v,c} = \sum_{v,c} b_{v,c} R_{v,c}$ implies $a_{v,c} = b_{v,c}$ for each $(v, c) \in \mathbb{F}^2$. The event of $\mathcal{O}$ not having output $\mathbf{V}$ means one of the following occurs.

1. The multi-set of counters $\{\mathbf{c}_i\}_{i \in [M]}$ doesn't equal $\{1, \dots, M\}$. In this case, the LHS of the third identity will not have all one coefficients for the elements $\{R_{0,i}\}_{i \in [M]}$ and so cannot equal the RHS.

Note that when we are not in this case the counters in $\mathcal{O}$ are all distinct, which we assume for the next cases.

2. For some $\mathbf{v}, \mathbf{c}_v, \mathbf{c}, (\text{del}, \mathbf{v}, \mathbf{c}_v, \mathbf{c}) \in \mathcal{O}$ but $(\text{add}, \mathbf{v}, 0, \mathbf{c}_v) \notin \mathcal{O}$; or for some $\mathbf{v}, \mathbf{c}_v, \mathbf{c}_1 \neq \mathbf{c}_2, (\text{del}, \mathbf{v}, \mathbf{c}_v, \mathbf{c}_1), (\text{del}, \mathbf{v}, \mathbf{c}_v, \mathbf{c}_2) \in \mathcal{O}$: In the first identity RHS, we will have $R_{\mathbf{v}, \mathbf{c}_v}$ with coefficient in the range $\{-1, \dots, -M\}$, while in the LHS it has coefficient one or zero.
3. For some $\mathbf{v}, \mathbf{c}_v, \mathbf{c}, (\text{read}, \mathbf{v}, \mathbf{c}_v, \mathbf{c}) \in \mathcal{O}$ but $(\text{add}, \mathbf{v}, 0, \mathbf{c}_v) \notin \mathcal{O}$: In the second identity RHS $R_{\mathbf{v}, \mathbf{c}_v}$ will have a coefficient in the range $\{1, \dots, M\}$ while in the LHS it has coefficient zero.
4. $\mathbf{V}$ is *not* equal to the set $\mathbf{V}'$ of $(\mathbf{v}, \mathbf{c})$ for which $(\text{add}, \mathbf{v}, 0, \mathbf{c}) \in \mathcal{O}$ but $(\text{del}, \mathbf{v}, \mathbf{c}, \mathbf{c}') \notin \mathcal{O}$. We look at the first identity. $\mathbf{V}'$ is precisely the set of $(\mathbf{v}, \mathbf{c})$ with coefficient one on the RHS, while $\mathbf{V}$ is the set of $(\mathbf{v}, \mathbf{c})$ with coefficient one on the LHS. Hence the first identity cannot hold in this case.

□

Defining $\mathcal{R}_{F^*, \ell^*}$ We now define the RCG relation $\mathcal{R}_{F^*, \ell^*}$ which will permit efficient proofs that record operations are consistent. The idea is to evaluate the above rational identities at a random point, and have the transition predicate incrementally check the evaluation is correct by a running sum. For the random evaluation point to be generated in a manner that is sound but also allow for an incremental computation of the rational identity summand by summand, we compute it via a hash chain. This chain includes at each step the new set of record operations and the last hash output, which is in essence a commitment to all the record operations in previous steps. This idea also appears in [Sou23, NDC⁺24, Gro24]. We proceed with the formal definition.

Let F denote the function $F(Z_F, W_F, Z_F^*, S_F) \rightarrow \{\text{accept}, \text{reject}\}$ from Section 4.1. Let $k := |S_F|$, $\text{cm} : \mathbb{F}^{5k} \rightarrow \mathbb{G}$, and set $M := kn$.

Define the function $F^* : (Z, W, Z^*) \rightarrow \{\text{accept}, \text{reject}\}$:

- $Z = (Z_F, h, \mathfrak{s}, \alpha, \beta, \varepsilon)$.
- $Z^* = (Z_F^*, h^*, \mathfrak{s}^*, \alpha^*, \beta^*, \varepsilon^*)$.
- $W = (W_F, S_F, m)$.

Under this notation $F^*(Z, W, Z^*) = \text{accept}$ if and only if

1. $F(Z_F, W_F, Z_F^*, S_F) = \text{accept}$.
2. If $Z_F.\text{init} = \text{true}$, $h = \emptyset$ and $\mathfrak{s} = 0$.
3. Let $S_F = \{(op_i, \mathbf{v}_i, \mathbf{c}_{\mathbf{v}_i}, \mathbf{c}_i)\}_{i \in [k]}$. We have $\mathbf{c}_{\mathbf{v}_i} < \mathbf{c}_i$ for all $i \in [k]$, and $\mathfrak{s}^* = \mathfrak{s} +$

$$\sum_{i \in [k]; op_i = \text{add}} \frac{1 + \varepsilon m_i}{\alpha + \beta \mathbf{v}_i + \mathbf{c}_i} - \sum_{i \in [k]; op_i = \text{read}} \frac{\varepsilon}{\alpha + \beta \mathbf{v}_i + \mathbf{c}_{\mathbf{v}_i}} - \sum_{i \in [k]; op_i = \text{del}} \frac{1}{\alpha + \beta \mathbf{v}_i + \mathbf{c}_{\mathbf{v}_i}} + \sum_{i \in [k]} \frac{\varepsilon^2}{\alpha + \mathbf{c}_i}.$$

4. $\alpha^* = \alpha, \beta^* = \beta, \varepsilon^* = \varepsilon$.
5. $h^* = \mathcal{H}(h, \text{cm}(S_F, m))$.

$\ell^*(Z, \mathbf{V}) = \text{accept}$ if and only if

1. $Z.g = g_{\text{empty}}$,
2. $\mathfrak{s} = \sum_{(\mathbf{v}, \mathbf{c}) \in \mathbf{V}} \frac{1}{\alpha + \beta \mathbf{v} + \mathbf{c}} + \sum_{i \in [M]} \frac{\varepsilon^2}{\alpha + i}$,
3. $\mathcal{H}(h, \mathbf{V}, 1) = \alpha, \mathcal{H}(h, \mathbf{V}, 2) = \beta, \mathcal{H}(h, \mathbf{V}, 3) = \varepsilon$.

We show that $\mathcal{R}_{F^*, \ell^*}$ captures $\mathcal{R}_{F, \ell}$ and consequently valid executions.

Lemma 5.2. *There is an efficiently computable map φ^* such that the following holds. Suppose an efficient adversary $\mathcal{A}$ outputs $(\phi, \omega) \in \mathcal{R}_{F^*, \ell^*}$. Set $d := 3M + |\mathbf{V}| + 1$. Define $\text{cm}^*(s_1, \dots, s_n, m_1, \dots, m_n) := h_n$, where $h_0 := \emptyset$ and $h_i := \mathcal{H}(h_{i-1}, \text{cm}(s_i, m_i))$ for $i \in [n]$. Let $\mathcal{D}$ be a function family to be defined in the proof. Assume the 3-variate ZTA holds for $(\mathcal{D}, \mathcal{H}, \text{cm}^*, d)$. Then e.w.p. $\text{negl}(\lambda)$, $\varphi^*(\phi, \omega) \in \mathcal{R}_{F, \ell}$.*

Proof. Given an instance $\phi = (z_{\text{final}}^*, C, \mathbf{V})$ with $z_{\text{final}}^* = (z_{\text{final}}, h, \delta, \alpha, \beta, \varepsilon)$ and witness $\omega = (n, z', w')$ with $z' = (z'_0, \dots, z'_n)$, $w' = (w'_1, \dots, w'_n)$ output by $\mathcal{A}$, denote $z'_i = (z_i, h_i, \delta_i, \alpha_i, \beta_i, \varepsilon_i)$, $w'_i = (w_i, s_i, m_i)$. Define $\varphi^*(\phi, \omega) := (x, w)$ as

$$x := (z_{\text{final}}, C, \mathbf{V}), \quad w := (n, (z_0, \dots, z_n), (w_1, \dots, w_n), (s_1, \dots, s_n)).$$

From $(\phi, \omega) \in \mathcal{R}_{F^*, \ell^*}$, we know w satisfies the transition constraints, namely for $i \in [n]$, $F(z_{i-1}, w_i, z_i, s_i) = \text{accept}$. We also know that $z_0.\text{init} = \text{true}$, $z_n = z_{\text{final}}$, and $n \leq C$. It is left to show that e.w.p. $\text{negl}(\lambda)$ $\ell(z_n, s_1, \dots, s_n, \mathbf{V}) = \text{accept}$. Since $z_n.g = \text{empty}$, this is equivalent to showing that the rational identities from Claim 5.1 hold e.w.p. $\text{negl}(\lambda)$.

From $F^*(z'_{i-1}, w'_i, z'_i) = \text{accept}$ for all $i \in [n]$ and $\ell^*(z'_n, \mathbf{V}) = \text{accept}$, we know the equations from Claim 5.1 hold at $\alpha, \beta, \varepsilon$. That is, defining the rational function

$$\begin{aligned} r(X, Y, Z) := & \sum_{(\mathbf{v}, \mathbf{c}) \in \mathbf{V}} \frac{1}{X + \mathbf{v}Y + \mathbf{c}} - \sum_{i \in [M], op_i = \text{add}} \frac{1}{X + \mathbf{v}_i Y + \mathbf{c}_i} + \sum_{i \in [M], op_i = \text{del}} \frac{1}{X + \mathbf{v}_i Y + \mathbf{c}_{\mathbf{v}_i}} \\ & + Z \left(\sum_{i \in [M], op_i = \text{add}} \frac{(m_1 \| \dots \| m_n)_i}{X + \mathbf{v}_i Y + \mathbf{c}_i} - \sum_{i \in [M], op_i = \text{read}} \frac{1}{X + \mathbf{v}_i Y + \mathbf{c}_{\mathbf{v}_i}} \right) \\ & + Z^2 \left(\sum_{i \in [M]} \frac{1}{X + \mathbf{c}_i} - \sum_{i \in [M]} \frac{1}{X + i} \right), \end{aligned}$$

we have $r(\alpha, \beta, \varepsilon) = 0$. If any of the three rational identities from Claim 5.1 does not hold, we have $r(X, Y, Z) \not\equiv 0$. Let $f \in \mathbb{F}_{\leq d}[X, Y, Z]$ denote the resulting non-zero degree- d polynomial when multiplying $r(X, Y, Z)$ with all of its denominators. Note that $f(\alpha, \beta, \varepsilon) = 0$. Define D as the function that computes $f(X, Y, Z)$ given $x := (s_1, \dots, s_n, m_1, \dots, m_n)$ and $\tau := \mathbf{V}$. Set $\mathcal{D} := \{D\}$. Then we can define an efficient adversary $\mathcal{A}'$ against the 3-variate ZTA for $(\mathcal{D}, \mathcal{H}, \text{cm}^*, d)$ that outputs the $(3, d)$ -relation (D, x, τ) , which has probability $\text{negl}(\lambda)$. $\square$

6 Non-interactive folding schemes

We fix a vector space K over $\mathbb{F}$, and an $\mathbb{F}$ -linear function $\text{cm} : \mathbb{F}^M \rightarrow K$ that will be an implicit parameter in Definition 6.1. We say a relation $\mathcal{R}$ is *cm-compatible* if every element of $\mathcal{R}$ has the form $(\text{cm}(\omega), \omega)$. We say $\mathcal{R}$ is *cm-extendable* if every element of $\mathcal{R}$ has the form $((\text{cm}(\omega), \tau), \omega)$ for some ω, τ .

Definition 6.1. Fix relations $\mathcal{R}$ and $\mathcal{R}_{\text{acc}}$ that are cm -compatible and cm -extendable, respectively. An $(\mathcal{R} \mapsto \mathcal{R}_{\text{acc}})$ -folding scheme is a pair of algorithms $(\mathbf{P}, \mathbf{V})$ such that

1. $\mathbf{P}$ on input $(\Phi, \phi'; \omega, \omega')$ produces a pair (Φ^*, ω^*) and proof π .
2. $\mathbf{V}$ on input $(\Phi, \phi', \Phi^*, \pi)$ outputs `accept` or `reject` such that
 - (a) **Completeness:** If $(\Phi, \omega) \in \mathcal{R}_{\text{acc}}$, $(\phi', \omega') \in \mathcal{R}$ and $\mathbf{P}(\Phi, \phi'; \omega, \omega') = (\Phi^*, \omega^*, \pi)$, then with probability $1 - \text{negl}(\lambda)$, $(\Phi^*, \omega^*) \in \mathcal{R}_{\text{acc}}$ and $\mathbf{V}(\Phi, \phi', \Phi^*, \pi) = \text{accept}$.
 - (b) **Knowledge soundness given extractable commitments:**
For any efficient $\mathcal{A}$ the probability of the following event is $\text{negl}(\lambda)$: $\mathcal{A}$ outputs $(\phi, \tau), \phi', (\phi^*, \tau^*), \omega, \omega', \omega^*, \pi$ such that
 - i. $\text{cm}(\omega) = \phi, \text{cm}(\omega') = \phi', \text{cm}(\omega^*) = \phi^*$,
 - ii. $\mathbf{V}(\phi, \phi', \phi^*, \pi) = \text{accept}$,
 - iii. $((\phi^*, \tau^*), \omega^*) \in \mathcal{R}_{\text{acc}}$,
 - iv. $((\phi, \tau), \omega) \notin \mathcal{R}_{\text{acc}}$ or $(\phi', \omega') \notin \mathcal{R}$.

Remark 6.2. The justification for requiring only knowledge soundness given extractable commitments is as follows. We assume the Algebraic Group Model [FKL18] and use a commitment scheme based on linear combination of group elements like [KZG10]. In this model with such a commitment scheme an adversary $\mathcal{A}$ must output ω , with $\text{cm}(\omega) = \phi$ whenever it outputs some $\phi \in K$. For more details see [FKL18] or Section 2.2 of [GWC19], as well as Section 7 of this paper.

6.1 Relations for folding schemes

We define a more general satisfiability relation than in [BC23, EG23]. We have as parameters, integers $n, \mathcal{M}, d$ and an $\mathbb{F}$ -vector space K . We have a

- *Constraint function* $f : \mathbb{F}^{\mathcal{M}} \rightarrow \mathbb{F}^n$ which is a vector of n $\mathcal{M}$ -variate polynomials of degree $\leq d$,
- *Instance predicate* $\mathbf{f} : K \rightarrow \{\text{accept}, \text{reject}\}$,
- *Commitment function* $\text{cm} : \mathbb{F}^{\mathcal{M}} \rightarrow K$ which is $\mathbb{F}$ -linear and assumed to be collision resistant.

Given $(f, \mathbf{f}, \text{cm})$ we define a relation $\mathcal{R}_{f, \mathbf{f}, \text{cm}}$ consisting of all pairs (ϕ, ω) such that

1. $f(\omega) = 0^n$.
2. $\mathbf{f}(\phi) = \text{accept}$.
3. $\phi = \text{cm}(\omega)$.

We refer to a relation of this form as a *protostar relation* as the definition is based on relations appearing in [BC23].

The relation $\mathcal{R}^{\text{rand}}$ For brevity, let $\mathcal{R} = \mathcal{R}_{f, \mathbf{f}, \text{cm}}$. As in [EG23], we define the “randomized relaxed” version of $\mathcal{R}$, $\mathcal{R}^{\text{rand}}$. First, some required notation. Let $t := \log n$. For $i \in [n]$, let $S_i \subseteq \{0, \dots, t-1\}$ be the set such that $i-1 = \sum_{j \in S_i} 2^j$. We define the t -variate polynomial pow_i as

$$\text{pow}_i(X_0, \dots, X_{t-1}) := \prod_{\ell \in S_i} X_\ell.$$

Note that if $\beta = (\beta, \beta^2, \beta^4, \dots, \beta^{2^{t-1}})$, $\text{pow}_i(\beta) = \beta^{i-1}$.

Given the above notation, $\mathcal{R}^{\text{rand}}$ consists of the pairs (Φ, ω) with $\Phi = (\phi, \beta, e)$ such that

1. $\phi = \text{cm}(\omega)$.
2. $\beta \in \mathbb{F}^t, e \in \mathbb{F}$ and we have

$$\sum_{i \in [n]} \text{pow}_i(\beta) f_i(\omega) = e.$$

(Here, f_i denotes the i -th output coordinate of f .)

6.2 The PROTOGALAXY scheme

Deviating from [EG23], we explicitly present PROTOGALAXY as a *non-interactive* folding scheme, and for the special case of folding $k = 1$ instances. We define $Z(X) := X \cdot (1 - X)$. We assume below $\mathcal{H}$ is a function mapping arbitrary strings to elements of $\mathbb{F}^*$.

$\mathbf{P}_{PG}(\Phi = (\phi, \beta, e), \phi_1; \omega, \omega_1)$:

1. Compute $\delta = \mathcal{H}(\Phi, \phi_1)$. Define $\delta := (\delta, \delta^2, \dots, \delta^{2^{t-1}}) \in \mathbb{F}^t$.
2. Compute the polynomial

$$F(X) := \sum_{i \in [n]} \text{pow}_i(\beta + X\delta) f_i(\omega).$$

(Note that $F(0) = \sum_{i \in [n]} \text{pow}_i(\beta) f_i(\omega) = e$.)

3. Denote the non-constant coefficients of F by $a := (F_1, \dots, F_t)$.
4. Compute $\alpha = \mathcal{H}(\Phi, \phi_1, a)$.
5. Compute $\beta^* \in \mathbb{F}^t$ where $\beta_i^* := \beta_i + \alpha \cdot \delta_i$.
6. Define the polynomial $G(X)$ as

$$G(X) := \sum_{i \in [n]} \text{pow}_i(\beta^*) f_i(X \cdot \omega + (1 - X)\omega_1).$$

7. Compute the polynomial $K(X)$ such that

$$G(X) = F(\alpha)X + Z(X)K(X).$$

8. Let $b := (K_0, \dots, K_{d-2})$ be the coefficients of $K(X)$.

9. Compute $\gamma = \mathcal{H}(\Phi, \phi_1, a, b)$.

10. Compute

$$e^* := F(\alpha)\gamma + Z(\gamma)K(\gamma).$$

Finally, output

- the instance $\Phi^* = (\phi^*, \beta^*, e^*)$, where

$$\phi^* := \gamma \cdot \phi + (1 - \gamma)\phi_1,$$

- the witness $\omega^* := \gamma \cdot \omega + (1 - \gamma) \cdot \omega_1$,
- and the proof $\pi := (a, b)$.

$$\underline{\mathbf{V}_{PG}^{\mathbf{f}, \mathbf{cm}}(\Phi, \phi_1, \Phi^*, \pi = (a, b))}:$$

1. Check that $\mathbf{f}(\phi_1) = \text{accept}$. Output reject otherwise.
2. Compute δ, α, γ as in the prover algorithm given Φ, ϕ_1, a, b .
3. Check that Φ^* is as computed as in the prover algorithm. Specifically, that
 - (a) For each $i \in [t]$, $\beta_i^* = \beta_i + \alpha \cdot \delta_i$.
 - (b) $e^* = F(\alpha)\gamma + Z(\gamma)K(\gamma)$, where the coefficients of F, K are defined by a, b respectively.
 - (c) $\phi^* = \gamma \cdot \phi + (1 - \gamma)\phi_1$.

Output **accept** iff this is the case.

Remark 6.3. We stress that the verifier's description depends on $\mathbf{f}, \mathbf{cm}$ (and $t = \log n$ which is implicitly defined by the former), but not f ; hence the notation $\mathbf{V}_{PG}^{\mathbf{f}, \mathbf{cm}}$. The independence of the verifier definition from f will be crucial in Section 8 to avoid a circular definition of F' .

For the knowledge soundness analysis we'll use a variant of the Zero-Testing Assumption from [LS24], see Definition 2.1.

Theorem 6.4. Set $d' := \max\{n, d\}$. Denote $\mathbf{cm}'((\omega, \beta, e), \omega_1) := ((\mathbf{cm}(\omega), \beta, e), \mathbf{cm}(\omega_1))$. Let $\mathcal{D}$ be a family of four functions to be defined in the proof. Assume that $\mathbf{cm}$ is collision resistant and the ZTA holds for $(\mathcal{D}, \mathcal{H}, \mathbf{cm}', d')$. Then **PROTOGALAXY** is a $(\mathcal{R} \mapsto \mathcal{R}^{\text{rand}})$ -folding scheme.

Proof. The main thing to prove is knowledge soundness given extractable commitments. Fix any efficient $\mathcal{A}$. Let E be the event that $\mathcal{A}$ outputs $\Phi = (\phi, \beta, e), \phi_1, \Phi^* = (\phi^*, \beta^*, e^*), \omega, \omega_1, \omega^*, \pi$ such that

1. $\text{cm}(\omega) = \phi, \text{cm}(\omega_1) = \phi_1, \text{cm}(\omega^*) = \phi^*,$
2. $\mathbf{V}_{PG}^{\mathbf{f}, \text{cm}}(\Phi, \phi_1, \Phi^*, \pi) = \text{accept},$
3. $(\Phi^*, \omega^*) \in \mathcal{R}^{\text{rand}},$
4. $(\Phi, \omega) \notin \mathcal{R}^{\text{rand}}$ or $(\phi_1, \omega_1) \notin \mathcal{R}.$

According to Definition 6.1, knowledge soundness is equivalent to E having probability $\text{negl}(\lambda)$ for any such $\mathcal{A}$. We construct an efficient $\mathcal{A}'$ that runs $\mathcal{A}$, and when E occurs outputs either a collision of cm or a degree d' -relation for $(\mathcal{H}, \text{cm})$. By the theorem assumption this implies E is contained in two events of probability $\text{negl}(\lambda)$, and must have probability $\text{negl}(\lambda)$ itself.

Assume we are in event E . Using linearity of cm , when E occurs we have

$$\text{cm}(\gamma\omega + (1 - \gamma)\omega_1) = \gamma\phi + (1 - \gamma)\phi_1 = \phi^* = \text{cm}(\omega^*).$$

Thus, if $\omega^* \neq \gamma\omega + (1 - \gamma)\omega_1$, $\mathcal{A}'$ can output $(\omega^*, \gamma\omega + (1 - \gamma)\omega_1)$ as a collision of cm . Now assume that $\omega^* = \gamma\omega + (1 - \gamma)\omega_1$. Suppose $\pi = (a, b)$, with $a = (a_1, \dots, a_t), b = (b_0, \dots, b_{d-2})$. Define $F_0(X) := e + \sum_{i \in [t]} a_i X^i, K'(X) := \sum_{i=0}^{d-2} b_i X^i$. Let $\delta, \delta, \alpha, \beta^*, \gamma$ be computed as in the prover description given a, b . Define the polynomials

$$\begin{aligned} F'(X) &:= F_0(X) - \sum_{i \in [n]} \text{pow}_i(\beta + X\delta) f_i(\omega), \\ G'(X) &:= F_0(\alpha)X + Z(X)K'(X) - \sum_{i \in [n]} \text{pow}_i(\beta^*) f_i(X\omega + (1 - X)\omega_1). \end{aligned}$$

Since $((\phi^*, \beta^*, e^*), \omega^*) \in \mathcal{R}^{\text{rand}}$ and $\mathbf{V}_{PG}^{\mathbf{f}, \text{cm}}(\Phi, \phi', \Phi^*, \pi) = \text{accept}$,

$$\begin{aligned} G'(\gamma) &= F_0(\alpha)\gamma + Z(\gamma)K'(\gamma) - \sum_{i \in [n]} \text{pow}_i(\beta^*) f_i(\gamma\omega + (1 - \gamma)\omega_1) \\ &= e^* - \sum_{i \in [n]} \text{pow}_i(\beta^*) f_i(\omega^*) = e^* - e^* = 0. \end{aligned}$$

Set $x := ((\omega, \beta, e), \omega_1)$ and $\tau_1 := (a, b)$. If $G' \not\equiv 0$, $\mathcal{A}'$ outputs the degree d' -relation (D_1, x, τ_1) , where D_1 is the function that computes $G'(X)$ given x, τ_1 .

Assume now that $G' \equiv 0$.

If $(\phi_1, \omega_1) \notin \mathcal{R}$, using $Z(0) = 0$ we have

$$G'(0) = - \sum_{i \in [n]} \text{pow}_i(\beta^*) f_i(\omega_1) = 0.$$

Define the polynomial $A(X) := \sum_{i \in [n]} f_i(\omega_1) \text{pow}_i(\beta_1 + X\delta, \beta_2 + X\delta^2, \dots, \beta_t + X\delta^{2^{t-1}})$. We have $A(\alpha) = 0$. Suppose first that $A(X) \not\equiv 0$. Then setting D_2 to be the function that computes $A(X)$ given x and $\tau_2 := a$, $\mathcal{A}'$ can output the degree n relation (D_2, x, τ_2) . Now assume $A(X) \equiv 0$. Define the polynomial

$$B(X, Y) := \sum_{i \in [n]} f_i(\omega_1) \text{pow}_i(\beta_1 + XY, \beta_2 + XY^2, \dots, \beta_t + XY^{2^{t-1}}).$$

We have that $B(X, \delta) \equiv 0$. Write $B(X, Y)$ as a polynomial in X over $\mathbb{F}[Y]$:

$$B(X) = \sum_{i=0}^t C_i(Y) X^i.$$

Because we're in the case $(\phi_1, \omega_1) \notin \mathcal{R}$, B is a combination of the n linearly independent polynomials $\left\{ \text{pow}_i(\beta_1 + XY, \beta_2 + XY^2, \dots, \beta_t + XY^{2^{t-1}}) \right\}_{i \in [n]}$ with at least one non-zero coefficient, and so $B(X, Y) \not\equiv 0$. This means one of the polynomials C_i is non-zero, while $C_i(\delta) = 0$. We can use this to let $\mathcal{A}'$ output the degree n relation (D_3, x, τ_3) , where D_3 is the function that computes C_i given x and $\tau_3 := \emptyset$.

Now, assume that $(\Phi, \omega) \notin \mathcal{R}^{\text{rand}}$. As we're still assuming $G' \equiv 0$, we have

$$G'(1) = F_0(\alpha) - \sum_{i \in [n]} \text{pow}_i(\beta^*) f_i(\omega) = 0.$$

But we also have $F'(\alpha) = G'(1)$ and so $F'(\alpha) = 0$. On the other hand,

$$F'(0) = e - \sum_{i \in [n]} \text{pow}_i(\beta) f_i(\omega) \neq 0.$$

Setting $\tau_4 := a$ and D_4 to be the function that computes F' given x, τ_4 , we have that (D_4, x, τ_4) is a degree $\log n$ relation that $\mathcal{A}'$ can output in this case. Setting $\mathcal{D} = \{D_1, D_2, D_3, D_4\}$ we have proven knowledge soundness under the theorem assumptions. $\square$

6.3 Representative protostar-relations

Definition 6.5. Given a relation $\mathcal{R}_0$ we say a protostar-relation $\mathcal{R} = \mathcal{R}_{f, \mathbf{f}, \text{cm}}$ represents $\mathcal{R}_0$ if there is an efficiently computable and invertible maps φ_0, φ_1 such that any efficient $\mathcal{A}$ has a $\text{negl}(\lambda)$ probability of outputting a pair (ϕ, ω) such that

1. $\varphi_0(\phi)$ is always of the form (ϕ, X) for some X .
2. Any efficient $\mathcal{A}$ has a $\text{negl}(\lambda)$ probability of outputting a pair (ϕ, ω) such that either:
 - (a) $(\phi, \omega) \in \mathcal{R}_0$ but $(\varphi_0(\phi), \varphi_1(\omega)) \notin \mathcal{R}$.

(b) $(\phi, \omega) \in \mathcal{R}$ but $(\varphi_0^{-1}(\phi), \varphi_1^{-1}(\omega)) \notin \mathcal{R}_0$.

We define the *size* of the protostar relation $R_{f, \mathbf{f}, \text{cm}}$ as $\mathcal{M} + n$.

Claim 6.6. *Assuming ZTA for a constant size family there is a protostar-relation $\mathcal{R}_{f, \mathbf{f}, \text{cm}}$ representing $\mathcal{R}_{\text{app}}$ of size $O(N + n)$.*

7 Adversaries supporting recursive extraction

Following [LS24], we define a model of “recursive extraction” for our analysis in Section 8. We assume our commitment function $\text{cm} : \mathbb{F}^{\mathcal{M}} \rightarrow K$ is surjective. We assume the existence of an efficiently computable injective representation function $R : K \rightarrow \mathbb{F}^{\mathcal{L}}$. Whenever an adversary $\mathcal{A}$ outputs $a \in K$ we assume it is represented as $R(a)$. When analyzing knowledge soundness of our zk-SNARK in the next section, we put the following limitation on $\mathcal{A}$. Say $\mathcal{A}$ outputs a vector v over $\mathbb{F}$. If there is an index i such that $(v_i, \dots, v_{i+\ell-1}) = R(a)$ for some $a \in K$, then $\mathcal{A}$ must also output $\omega \in \mathbb{F}^{\mathcal{M}}$ such that $\text{cm}(\omega) = a$. We call such $\mathcal{A}$ a *recursive algebraic adversary*.

This assumption is motivated by a “recursive” interpretation of the Algebraic Group Model [FKL18] as done in [LS24]. For illustration, consider first the case where $\text{cm} : \mathbb{F}^n \rightarrow \mathbb{G}$ is defined as $\text{cm}(\omega) = \langle \omega, V \rangle$ for a fixed vector $V \in \mathbb{G}^n$ derived in a setup procedure. I.e. cm is a Pedersen commitment in this case. In this case, the AGM forces $\mathcal{A}$ when outputting $a \in \mathbb{G}$ to also output ω such that $\langle \omega, V \rangle = \text{cm}(\omega) = a$. [LS24] now raise the idea that if a prespecified subset of indices of ω is a valid representation of $b \in \mathbb{G}$, it is reasonable to demand of an algebraic $\mathcal{A}$ to also output ω_2 with $\langle \omega_2, V \rangle = b$.

Our concrete choice for cm when using PROTOGALAXY, is of the following form.² We have a setup procedure outputting a vector of group elements $V \in \mathbb{G}^n$ as output. We have some fixed partition of our input $\omega \in \mathbb{F}^{\mathcal{M}}$ into continuous segments of size either one or n . To get $\text{cm}(\omega)$ we operate segment-wise. If the segment ω_i is of size one we simply append ω_i to the output. If a segment $s = (\omega_i, \dots, \omega_{i+n-1})$ is of size n we append $\langle s, V \rangle$ to the output. In particular the output $\text{cm}(\omega)$ is a mixture of $\mathbb{F}$ and $\mathbb{G}$ elements, and accordingly the space K is a direct sum of spaces $\mathbb{V}_i \in \{\mathbb{F}, \mathbb{G}\}$. It follows that an algebraic adversary outputting $c \in K$ must output ω with $\text{cm}(\omega) = c$ as the AGM forces it to send an opening $s \in \mathbb{F}^n$ to each $a \in \mathbb{G}$.

8 The main construction

We say an RCG relation $\mathcal{R}_{F, \ell}$ is *trivial* if the S variable is not used in F , and accordingly ℓ depends only on (z_{final}, V) . We assume in this section $\mathcal{R}_{F, \ell}$ is trivial, thinking of it as $\mathcal{R}_{F^*, \ell^*}$ from Section 5.

² cm is of this form because it is derived from an interactive protocol where the prover messages are in committed form, but verifier challenges are in the clear. See [BC23, EG23] for more details.

8.1 The extended function F'

As in [KST21, BCL⁺21], we first describe an extended function F' that executes the folding verifier in addition to an iteration of F . Loosely speaking, extending F into F' is what enables bootstrapping a folding scheme into an IVC or RCG. We describe the function arguments first.

$$\mathfrak{x} = (z, \text{count}, h)$$

- z – output for F
- count – counter of IVC step
- h – hash of accumulator

$$\mathfrak{w} = (\text{acc}, \text{acc}_0, \mathfrak{x}_0, w, \pi)$$

- acc – current accumulator instance
- acc_0 – previous accumulator instance
- $\mathfrak{x}_0$ – instance (of F') to be accumulated
- w – private input for F
- π – proof for **PROTOGALAXY** verifier

We fix $\mathbf{f}, \mathbf{cm}, \varphi_1, \varphi_2$ to be the values appearing in the statement of Claim 6.6. $F'(\mathfrak{x}, \mathfrak{w}) = \text{accept}$ if and only if:

1. $F(\mathfrak{x}_0.z, w, z) = \text{accept}$.
2. If $\text{count} = 1$:
 - $\mathfrak{x}_0.z.\text{init} = \text{true}$.
3. If $\text{count} > 1$:
 - (a) $\mathcal{H}(\text{acc}) = h$.
 - (b) $\mathcal{H}(\text{acc}_0) = \mathfrak{x}_0.h$.
 - (c) $\mathfrak{x}_0.\text{count} = \text{count} - 1$.
 - (d) $\mathbf{V}_{PG}^{\mathbf{f}, \mathbf{cm}}(\text{acc}_0, \varphi_0(\mathfrak{x}_0), \text{acc}, \pi) = \text{accept}$.

Let $\mathcal{R}_0$ be the set of pairs $(\mathfrak{x}, \mathfrak{w})$ with $F'(\mathfrak{x}, \mathfrak{w}) = \text{accept}$. Let $\mathcal{R}$ be a protostar-relation representing $\mathcal{R}_0$ as defined in Section 6.3.

For readability, from now on we denote an accumulator by acc , accumulator witness by $\mathbf{w}\text{-acc}$, instance by inst , and instance witness by $\mathbf{w}\text{-inst}$.

8.2 The relation $\mathcal{R}_{\text{fin}}$

We define the relation $\mathcal{R}_{\text{fin}}$ of pairs (x, w) with

- $x = (\text{acc}, z_{\text{final}}, C, V)$
- $w = (\mathbf{w}\text{-acc}, \text{acc}_0, \text{inst}, \pi)$

such that

1. $(\text{acc}, \mathbf{w}\text{-acc}) \in \mathcal{R}^{\text{rand}}$.
2. $\mathbf{V}_{PG}^{\text{f}, \text{cm}}(\text{acc}_0, \text{inst}, \text{acc}, \pi) = \text{accept}$.
3. $\text{inst}.z = z_{\text{final}}$.
4. $\text{inst.count} \leq C$.
5. $\text{inst}.h = \mathcal{H}(\text{acc}_0)$.

Let $(\mathbf{P}_{\text{fin}}, \mathbf{V}_{\text{fin}})$ be a zk-SNARK for $\mathcal{R}_{\text{fin}}$.

8.3 Main construction

We now describe the full prover and verifier for a given trivial RCG relation $\mathcal{R}_{F, \ell}$. Later we review how to use this to get a zk-SNARK for the relation $\mathcal{R}_{\text{exec}}$ of valid executions, given the reductions of the previous sections.

$\mathbf{P}(x, w)$:

1. Let $x = (z_{\text{final}}, C, V)$, $w = (n, (z_0, \dots, z_n), (w_1, \dots, w_n))$. Recall that $(x, w) \in \mathcal{R}_{F, \ell}$ implies $F(z_{i-1}, w_i, z_i) = \text{accept}$ for each $i \in [n]$, $z_0.\text{init} = \text{true}$, $z_n = z_{\text{final}}$, $n \leq C$ and $\ell(z_{\text{final}}, V) = \text{accept}$.
2. Choose instance $\text{inst}_0 = (z_0, \text{count}_0, h_0)$ for arbitrary values count_0, h_0 . Choose acc_0 arbitrarily. Let $\text{inst}_1 = (z_1, 1, h_1)$, $\mathbf{w}\text{-inst}_1 = (\text{acc}_0, \text{acc}_0, \text{inst}_0, w_1, \pi_0)$ for arbitrary values h_1, acc_0, π_0 . (We can choose some values arbitrarily as they aren't constrained in F' when $\text{count} = 1$.)
3. Let acc_1 be a randomly chosen satisfiable accumulator. Namely, $\text{acc}_1 = (\text{cm}(\mathbf{w}\text{-acc}_1), \beta, e)$ where $\mathbf{w}\text{-acc}_1$ and β are chosen randomly³, and e is set to $e = \sum_{i \in [n]} \text{pow}_i(\beta) f_i(\mathbf{w}\text{-acc}_1)$.
4. For each $2 \leq i \leq n$, compute
 - (a) $(\text{acc}_i, \mathbf{w}\text{-acc}_i, \pi_i) = \mathbf{P}_{\text{cm}, n, f}^{\text{PG}}(\text{acc}_{i-1}, \mathbf{w}\text{-acc}_{i-1}, \text{inst}_{i-1}, \mathbf{w}\text{-inst}_{i-1})$
 - (b) $\text{inst}_i = (z_i, i, \mathcal{H}(\text{acc}_i))$,

³We only need $\text{cm}(\mathbf{w}\text{-acc}_1)$ to be uniformly distributed over the image of cm . According to cm 's structure, it may suffice to choose only a small subset of $\mathbf{w}\text{-acc}_1$'s coordinates randomly, and set the rest to zero.

- (c) $\mathbf{w-inst}_i = (\text{acc}_i, \text{acc}_{i-1}, \text{inst}_{i-1}, w_i, \pi_i)$
- 5. $(\text{acc}, \mathbf{w-acc}, \pi^*) = \mathbf{P}_{\text{cm}, n, f}^{\text{PG}}(\text{acc}_n, \mathbf{w-acc}_n, \text{inst}_n, \mathbf{w-inst}_n)$.
- 6. Let $\pi_{\text{fin}} = \mathbf{P}_{\text{fin}}(x, w)$ where
 - $x = (\text{acc}, V, C, z_{\text{final}})$
 - $w = (\mathbf{w-acc}, \text{acc}_n, \text{inst}_n, \pi^*)$
- 7. Output $\pi = (\text{acc}, \pi_{\text{fin}})$.

$\mathbf{V}(x, \pi)$:

1. Parse x as (z_{final}, C, V) , π as $(\text{acc}, \pi_{\text{fin}})$.
2. If $\mathcal{J}(z_{\text{final}}, V) = \text{reject}$ output reject.
3. Let $x := (\text{acc}, z_{\text{final}}, C, V)$.
4. Return $\mathbf{V}_{\text{fin}}(x, \pi_{\text{fin}})$.

Theorem 8.1. *Let $\mathcal{R}_{F, \ell}$ be a trivial RCG relation. Assume that $\mathcal{H}$ is collision resistant, and that the assumptions in Theorem 6.4 hold. Then $(\mathbf{P}, \mathbf{V})$ is a space-efficient zk-SNARK for $\mathcal{R}_{F, \ell}$.*

Proof. The main thing to prove is knowledge soundness. Let $\mathcal{A}$ be a recursive algebraic adversary.

We define the following extractor algorithm $\mathbf{E}$:

1. Given (x, π) , use the SNARK extractor to output $w = (\mathbf{w-acc}, \text{acc}^*, \text{inst}^*, \pi^*)$.
2. Let $n := \text{inst}^*. \text{count}$. Denote $\text{acc}_n := \text{acc}^*, \text{inst}_n := \text{inst}^*$.
3. If $(x, w) \notin \mathcal{R}_{\text{fin}}$ output *empty* and abort. Otherwise $\text{acc}_n, \text{inst}_n \in K$ and let $\mathbf{w-acc}_n, \mathbf{w-inst}_n$ be the elements output by $\mathcal{A}$ such that $\text{cm}(\mathbf{w-acc}_n) = \text{acc}_n$ and $\text{cm}(\mathbf{w-inst}_n) = \text{inst}_n$.
4. For $i = n - 1, \dots, 1$:
 - (a) If $(\text{inst}_{i+1}, \mathbf{w-inst}_{i+1}) \notin \mathcal{R}$, output *empty* and abort. Otherwise, as $(\text{inst}_{i+1}, \mathbf{w-inst}_{i+1}) \in \mathcal{R}$ implies $\text{acc}_i, \text{inst}_i \in K$, let $\mathbf{w-acc}_i, \mathbf{w-inst}_i$ be the elements output by $\mathcal{A}$ such that $\text{acc}_i = \text{cm}(\mathbf{w-acc}_i)$, $\text{inst}_i = \text{cm}(\mathbf{w-inst}_i)$.
 - (b) Parse inst_i as $(z_i, \text{count}_i, h_i)$. Parse $\mathbf{w-inst}_i$ as $(\text{acc}'_i, \text{acc}_{i-1}, \text{inst}_{i-1}, w_i, \pi_i)$. Note that through this parsing we have in particular defined z_i, w_i and inst_{i-1} .
5. Define $z_0 := \text{inst}_0.z$.
6. Output $w := (n, (z_0, \dots, z_n), (w_1, \dots, w_n))$.

Let D be the event that $\mathcal{A}$ outputs (x, π) such that $\mathbf{V}(x, \pi) = \text{accept}$. If D has probability $\text{negl}(\lambda)$ we are done. Otherwise, in order to prove knowledge soundness, we need to show that

$$\Pr[(x, w) \notin \mathcal{R}_{F, \ell} \mid D] = \text{negl}(\lambda).$$

Assume from now we are in the space conditioned on D . Let E_0 be the event $(x, w) \notin \mathcal{R}_{\text{fin}}$. For $i \in [n]$, let E_i be the union of the following events:

1. $(\text{inst}_i, w\text{-inst}_i) \notin \mathcal{R}$.
2. $(\text{acc}_i, w\text{-acc}_i) \notin \mathcal{R}$.
3. $\mathcal{H}(\text{acc}_i) \neq \text{inst}_i.h$.

We first argue that if $E_0, \dots, E_n$ don't occur, $(x, w) \in \mathcal{R}_{F, \ell}$. Denote $x = (z_{\text{final}}, C, \mathbf{V})$ and $w = (\text{acc}, x)$. Examining the definition of $\mathcal{R}_{F, \ell}$, the statement $(x, w) \in \mathcal{R}_{F, \ell}$ is equivalent to the following conditions.

1. $\ell(z_{\text{final}}, \mathbf{V}) = \text{accept}$: $\mathbf{V}$ checks this directly so we know it holds.
2. $\text{inst}_n.z = z_{\text{final}}$ and $n \leq C$: Follows from $(x, w) \in \mathcal{R}_{\text{fin}}$.
3. For all $i \in [n]$ $F(z_{i-1}, w_i, z_i) = \text{accept}$: Follows directly from $(\text{inst}_i, w\text{-inst}_i) \in \mathcal{R}$ for each $i \in [n]$.
4. $z_0.\text{init} = \text{true}$: For this we first prove by backwards induction on $i = n, n-1, \dots, 1$ that $\text{inst}_i.\text{count} = i$. By our definition of inst_n this holds for $i = n$. For the induction step, assume it holds for i . Since $(\text{inst}_i, w\text{-inst}_i) \in \mathcal{R}$ we have $\text{inst}_{i-1}.\text{count} = w\text{-inst}_{i-1}.\text{inst}.\text{count} = \text{inst}_i.\text{count} - 1 = i - 1$.

In particular, $\text{inst}_1.\text{count} = 1$ which implies when $(\text{inst}_1, w\text{-inst}_1) \in \mathcal{R}$ that $z_0.\text{init} = \text{inst}_0.z.\text{init} = \text{true}$.

It is left to show that the probability of $E_0 \cup E_1 \cup \dots \cup E_n$ is $\text{negl}(\lambda)$. From the knowledge-soundness of the zk-SNARK for $\mathcal{R}_{\text{fin}}$ we have $\Pr[E_0] = \text{negl}(\lambda)$. (Recall we are in the event D and assuming its probability is non-negligible.)

For this purpose, we prove by backwards induction on i , for each $i \in [n]$, that the probability that E_i occurs given $E_0, E_{i+1}, \dots, E_n$ didn't is $\text{negl}(\lambda)$.

For $i = n$, assume E_0 didn't occur, hence $(x, w) \in \mathcal{R}_{\text{fin}}$. In this case $(\text{acc}, w\text{-acc}) \in \mathcal{R}^{\text{rand}}$ and $\mathbf{V}_{PG}^{\text{f, cm}}(\text{inst}_n, \text{acc}_n, \text{acc}, \pi^*) = \text{accept}$. Define $\mathcal{A}_n$ to be the algorithm that executes $\mathbf{E}$ and outputs $\text{acc}_n, \text{inst}_n, \text{acc}, w\text{-acc}_n, w\text{-inst}_n, w\text{-acc}, \pi^*$. From the knowledge soundness of **PROTOGALAXY** applied to $\mathcal{A}_n$ we have that the probability conditioned on $\neg E_0$ that E_n occurred is $\text{negl}(\lambda)$.

Now assume the induction hypothesis for some $i > 1$. We have by the hypothesis that the event $D_i := \neg(E_0 \cup E_i \cup \dots \cup E_n)$ has probability $1 - \text{negl}(\lambda)$. Assume that we are in D_i . From $(\text{inst}_i, w\text{-inst}_i) \in \mathcal{R}$ we have that $\text{inst}_i.h = \mathcal{H}(\text{acc}'_i) = \mathcal{H}(\text{acc}_i)$. From the collision resistance of $\mathcal{H}$, we thus have that the probability that $\text{acc}_i \neq \text{acc}'_i$ is $\text{negl}(\lambda)$. If $\text{acc}_i = \text{acc}'_i$ we have from $(\text{inst}_i, w\text{-inst}_i) \in \mathcal{R}$ that $\mathbf{V}(\text{acc}_{i-1}, \text{inst}_{i-1}, \pi_i, \text{acc}_i) = \text{accept}$. Define

$\mathcal{A}_i$ to be the algorithm that executes $\mathbf{E}$ and outputs $\text{acc}_{i-1}, \text{inst}_{i-1}, \text{w-acc}_{i-1}, \text{w-inst}_{i-1}, \text{w-acc}_i, \pi_i$. From the knowledge soundness of **PROTOGALAXY** applied to $\mathcal{A}_i$ we have that the probability that $(\text{inst}_{i-1}, \text{w-inst}_{i-1}) \notin \mathcal{R}$ or $(\text{acc}_{i-1}, \text{w-acc}_{i-1}) \notin \mathcal{R}^{\text{rand}}$ given D_i is $\text{negl}(\lambda)$. Noting that $(\text{inst}_i, \text{w-inst}_i) \in \mathcal{R}$ also implies $\mathcal{H}(\text{acc}_{i-1}) = \text{inst}_{i-1}.h$, $\Pr[E_{i-1} \mid D_i] = \text{negl}(\lambda)$; which is the desired inductive statement. $\square$

8.4 Constructing proofs for $\mathcal{R}_{\text{exec}}$

Going through the reductions of the previous sections, we get the following zk-SNARK for the relation $\mathcal{R}_{\text{exec}}$ from Section 3.3 describing valid executions.

$\mathbf{P}_{\text{exec}}(\mathbf{x}_{\text{exec}}, \mathbf{w}_{\text{exec}})$:

1. Let $\mathbf{x}_{\text{exec}} = (\mathbf{r}, C, \mathbf{V})$, $z_{\text{final}} = (g_{\text{empty}}, \mathbf{r}, \text{false})$ and $x = (z_{\text{final}}, C, \mathbf{V})$. Let φ be the map in Lemma 4.1, and compute $\mathbf{w} = \varphi(\mathbf{w}_{\text{exec}})$.
2. Let $\mathbf{w} = (n, (z_0, \dots, z_n), (w_1, \dots, w_n), (s_1, \dots, s_n))$.
 - (a) For each $i \in [n]$, define $m_i \in \mathbb{F}^k$ such that $m_{i,j}$ is the number of times a record is read when $s_{i,j}$ is an **add** operation, and 0 otherwise.
 - (b) Set $h_0 = \emptyset$ and compute $h_i = \mathcal{H}(h_{i-1}, \text{cm}(s_i, m_i))$ for $i \in [n]$.
 - (c) Compute $(\alpha, \beta, \varepsilon) = (\mathcal{H}(h_n, \mathbf{V}, 1), \mathcal{H}(h_n, \mathbf{V}, 2), \mathcal{H}(h_n, \mathbf{V}, 3))$.
 - (d) Set $s_0 = 0$ and compute s_i for $i \in [n]$ as in the definition of F^* in Section 5.
 - (e) Let $z_{\text{final}}^* = (z_{\text{final}}, h_n, s_n, \alpha, \beta, \varepsilon)$, for $0 \leq i \leq n$, let $z_i^* = (z_i, h_i, s_i, \alpha, \beta, \varepsilon)$, and for $i \in [n]$, let $w_i^* = (w_i, s_i, m_i)$.
 - (f) Set $x^* = (z_{\text{final}}^*, C, \mathbf{V})$ and $\mathbf{w}^* = (n, (z_0^*, \dots, z_n^*), (w_1^*, \dots, w_n^*))$.
3. Compute the proof π^* for $(x^*, \mathbf{w}^*) \in \mathcal{R}_{F^*, \ell^*}$ according to $\mathbf{P}$ in Section 8.3.
4. Output $\pi = (h_n, s_n, \alpha, \beta, \varepsilon, \pi^*)$.

$\mathbf{V}_{\text{exec}}(\mathbf{x}_{\text{exec}}, \pi)$:

1. Parse $\mathbf{x}_{\text{exec}}$ as $(\mathbf{r}, C, \mathbf{V})$ and π as $(h, s, \alpha, \beta, \varepsilon, \pi^*)$. Let $z_{\text{final}} = (g_{\text{empty}}, \mathbf{r}, \text{false})$, $z_{\text{final}}^* = (z_{\text{final}}, h, s, \alpha, \beta, \varepsilon)$ and $x^* = (z_{\text{final}}^*, C, \mathbf{V})$.
2. Return $\mathbf{V}(x^*, \pi^*)$ according to $\mathbf{V}$ in Section 8.3.

Theorem 8.2. *If $(\mathbf{P}, \mathbf{V})$ from Section 8.3 is knowledge-sound for $\mathcal{R}_{F^*, \ell^*}$ and the necessary assumptions from Lemma 5.2 hold, $(\mathbf{P}_{\text{exec}}, \mathbf{V}_{\text{exec}})$ is knowledge-sound for $\mathcal{R}_{\text{exec}}$.*

Proof. Let $\mathcal{A}$ be a recursive algebraic adversary that outputs $\mathbf{x}_{\text{exec}} = (\mathbf{r}, C, \mathbf{V})$ and $\pi = (h, s, \alpha, \beta, \varepsilon, \pi^*)$. Let $z_{\text{final}} = (g_{\text{empty}}, \mathbf{r}, \text{false})$, $z_{\text{final}}^* = (z_{\text{final}}, h, s, \alpha, \beta, \varepsilon)$, and $x^* = (z_{\text{final}}^*, C, \mathbf{V})$. Define $\mathcal{A}^*$ as the adversary that runs $\mathcal{A}$ but outputs (x^*, π^*) . Define the following extractor $\mathbf{E}_{\text{exec}}$:

1. Given $(\mathbf{x}_{\text{exec}}, \pi)$, use $\mathbf{E}$ from Section 8.3 to extract $\mathbf{w}^*$ from the adversary $\mathcal{A}^*$.

2. Compute $(x, w) = \varphi^*(x^*, w^*)$, where φ^* is the map from Lemma 5.2.
3. Output $w_{\text{exec}} = \varphi^{-1}(w)$, where φ is the map from Lemma 4.1.

We need to show that

$$\Pr[\mathbf{V}_{\text{exec}}(x_{\text{exec}}, \pi) = \text{accept} \wedge (x_{\text{exec}}, w_{\text{exec}}) \notin \mathcal{R}_{\text{exec}}] = \text{negl}(\lambda).$$

If the first event in the above expression has negligible probability, we are done.

Otherwise, assuming we are in the first event, i.e., $\mathbf{V}_{\text{exec}}$ accepts, we show the second event has negligible probability. In particular, this means $\mathbf{V}(x^*, \pi^*) = \text{accept}$.

From the knowledge soundness of $(\mathbf{P}, \mathbf{V})$ for $\mathcal{R}_{F^*, \ell^*}$, we get that the witness w^* output by $\mathbf{E}$ satisfies $(x^*, w^*) \in \mathcal{R}_{F^*, \ell^*}$ e.w.p. $\text{negl}(\lambda)$.

From Lemma 5.2, we get $(x, w) \in \mathcal{R}_{F, \ell}$ e.w.p. $\text{negl}(\lambda)$.

From Lemma 4.1, we get $(x_{\text{exec}}, w_{\text{exec}}) \in \mathcal{R}_{\text{exec}}$ e.w.p. $\text{negl}(\lambda)$, finishing the proof. $\square$

9 General Final predicate

We sketch how to deal with a *general* final predicate ℓ , rather than the one described so far for checking record operations. A more detailed explanation will appear in an updated version of the paper.

The basic idea is that a general final predicate reduces to the concatenation predicate. Namely, suppose we can prove an element s is the commitment to the concatenation of the $\{s_i\}$ from all n iterations. Then we can modify the final zk-SNARK in Section 8 to also check that $\ell(z_{\text{final}}, s_1, \dots, s_n, \mathbf{V}) = \text{accept}$.

For this purpose, instead of the function F^* in Section 5 we add **PROTOGALAXY** constraints to the transition function F to obtain at iteration i a vector S_i that is the concatenation of S_{i-1} with s_i .

9.1 Efficient Proofs of Concatenation with PROTOGALAXY

Given a sequence consisting of vectors $\{s_i\}_{i \in [n]}$ of field elements of length k and a bound C on n , we can define a system of linear protogalaxy constraints that a concatenation was correctly carried out. We highlight that it is important the constraint system does not depend on the current iteration i , or actual sequence length n , only on the bound C . The idea is that if we append the new elements *at the beginning* of the vector, the concatenation constraints can be written as linear constraints that indeed only depend on k, C but not i or n .

Fix vector $b \in \mathbb{F}^k$ and vectors $a, a' \in \mathbb{F}^{kC}$. For each $j \in [kC]$ we define the constraint $f_j(a, b, a')$ as follows.

$$f_j(a, b, a') := \begin{cases} a'_j - b_j & i \leq k \\ a'_j - a_{j-k} & j > k \end{cases}.$$

In words, the constraints $\{f_j\}$ enforce that a' consists of the k elements of b followed by the first $(C-1)k$ elements of a . Additionally, we will constrain (using the init variable)

that S_0 is the zero vector of length kC . In the i -th iteration, our transition function F will check for each $j \in [kC]$ the constraint $f_j(S_i, s_i, S_{i+1}) = 0$.

One can now show inductively that when starting from the zero vector, under the assumptions that $n \leq C$ and $|s_i| \leq k$ for each $i \in [n]$, we end up with $S_n = (s_1 \| \dots \| s_n)$ possibly padded with zeroes.

Prover efficiency If the stack is folded in order, then even in the new randomized instance, the zero padding is preserved. That is, higher order elements above ki are still zero after folding. So the prover only needs to perform one field multiplication per stack element to compute the folded witness. These constraints are also linear so they do not contribute to any of the error terms in the $G(X)$ polynomial.

For all $j > k(i+1)$ notice that $f_j(S_i, s_i, S_{i+1}) = 0$ identically, so they do not even contribute to the cost of computing the $F(X)$ polynomial. Assuming all of these constraints are organized adjacently in the highest order indices of the overall protogalaxy constraint system, there is a straightforward modification of the $F(X)$ computation which incurs no cost for constraints that are identically zero. In brief, if an entire subtree consists of leaves that are zero, then the prover can avoid any computation for that subtree. Therefore, the total prover computation depends only on the size of the vector S_{i+1} .

9.2 RCG and Goblin $\mathcal{P}\text{IonK}$

The “Global” in RCG suggests a final predicate checking conditions *between* the different s_i . However, as suggested in [Wil23], it might

Acknowledgements

The third author was funded by the Vienna Science and Technology Fund (WWTF) [10.47379/VRG18002] and by the Austrian Science Fund (FWF) [10.55776/F8515-N].

References

- [BC23] B. Bünz and B. Chen. Protostar: Generic efficient accumulation/folding for special sound protocols. *IACR Cryptol. ePrint Arch.*, page 620, 2023.
- [BCCT12] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer. Recursive composition and bootstrapping for SNARKs and proof-carrying data. Cryptology ePrint Archive, Paper 2012/095, 2012. <https://eprint.iacr.org/2012/095>.
- [BCL⁺21] B. Bünz, A. Chiesa, W. Lin, P. Mishra, and N. Spooner. Proof-carrying data without succinct arguments. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 681–710. Springer, 2021.

- [CT10] A. Chiesa and E. Tromer. Proof-carrying data and hearsay arguments from signature cards. In *Innovations in Computer Science - ICS 2010, Tsinghua University, Beijing, China, January 5-7, 2010. Proceedings*, pages 310–331. Tsinghua University Press, 2010.
- [Eag22] Liam Eagen. Bulletproofs++. *IACR Cryptol. ePrint Arch.*, page 510, 2022.
- [EG23] Liam Eagen and Ariel Gabizon. ProtoGalaxy: Efficient ProtoStar-style folding of multiple instances. Cryptology ePrint Archive, Paper 2023/1106, 2023. <https://eprint.iacr.org/2023/1106>.
- [FKL18] G. Fuchsbauer, E. Kiltz, and J. Loss. The algebraic group model and its applications. In *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part II*, pages 33–62, 2018.
- [Gro24] J. Groth. Memory checking in folding-based zkvm - jens groth (nexus), 2024. https://www.youtube.com/watch?v=xnzjC_9vUzs.
- [GWC19] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. PLONK: permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *IACR Cryptology ePrint Archive*, 2019:953, 2019.
- [Hab22] U. Haböck. Multivariate lookups based on logarithmic derivatives. *IACR Cryptol. ePrint Arch.*, page 1530, 2022.
- [KST21] A. Kothapalli, S. T. V. Setty, and I. Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. *IACR Cryptol. ePrint Arch.*, page 370, 2021.
- [KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. Constant-size commitments to polynomials and their applications. pages 177–194, 2010.
- [LS24] H. Lee and J. H. Seo. On the security of nova recursive proof system. Cryptology ePrint Archive, Paper 2024/232, 2024. <https://eprint.iacr.org/2024/232>.
- [NDC⁺24] W. Nguyen, T. Datta, B. Chen, N. Tyagi, and D. Boneh. Mangrove: A scalable framework for folding-based SNARKs. Cryptology ePrint Archive, Paper 2024/416, 2024. <https://eprint.iacr.org/2024/416>.
- [Sou23] L. Soukhanov. Folding endgame, 2023. <https://zkresear.ch/t/folding-endgame/106>.
- [Val08] P. Valiant. Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In *Theory of Cryptography, Fifth Theory of Cryptography Conference, TCC 2008, New York, USA, March 19-21, 2008*, volume 4948 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2008.

- [Wil23] Z. J. Williamson. Goblin plonk: lazy recursive proof composition, 2023.
<https://hackmd.io/@aztec-network/B19AA8812>.